

南開大學
Nankai University

计算机学院

《人工智能导论》课程实验报告

斑马问题

姓名：赵宇哲

学号：2411094

专业：计算机科学与技术

实验时间：2025 年 10 月 13 日 星期一

目录

1 问题重述 **2**

2 设计思想 **3**

2.1 导入 kanren 包 3
2.2 辅助函数设计 3
2.3 智能体构建 3
2.4 逻辑规则声明 4
2.4.1 包含关系 membero 5
2.4.2 等价关系 eq 5
2.4.3 和或关系 conde 5
2.5 规则求解器 5

3 程序代码 **6**

4 测试结果 **8**

5 总结 **8**

1 问题重述

斑马问题： 5 个不同国家（英国、西班牙、日本、意大利、挪威）且工作各不相同（油漆工、摄影师、外交官、小提琴家、医生）的人分别住在一条街上的 5 所房子里，每所房子的颜色不同（红色、白色、蓝色、黄色、绿色），每个人都有自己养的不同宠物（狗、蜗牛、斑马、马、狐狸），喜欢喝不同的饮料（矿泉水、牛奶、茶、橙汁、咖啡）。根据以下提示，得出哪所房子里的人养斑马，哪所房子里的人喜欢喝矿泉水。

1. 英国人住在红色的房子里
2. 西班牙人养了一条狗
3. 日本人是一名油漆工
4. 意大利人喜欢喝茶
5. 挪威人住在左边的第一间房子里
6. 绿色房子在白房子的右边
7. 摄影师养了一只蜗牛
8. 外交官住在黄房子里
9. 住在中间那间房子的人喜欢喝牛奶
10. 喜欢喝咖啡的人住在绿房子里
11. 挪威人住在蓝色的房子旁边
12. 小提琴家喜欢喝橙汁
13. 养狐狸的人的房子与医生的房子相邻
14. 养马的人的房子与外交官的房子相邻

题目给出一排 5 间房子，每间房子有如下五类属性：国家、颜色、职业、宠物、饮料。

问题解读

同一类属性在五间房子中各不相同，因此每一类属性都与房子形成一一对应关系。我们的任务是依据题目提供的约束条件，推理出每间房子的主人所属国家、职业、房屋颜色、所养宠物及其所喝饮料的完整对应关系。

2 设计思想

根据题目要求，我们需要利用 kanren 库提供的声明式逻辑求解机制，将题目条件抽象为一系列约束关系，由解释器自动推导出满足全部条件的唯一解。

2.1 导入 kanren 包

在运行程序前，需在 CMD 执行以下命令行实现 kanren 包的安装：

```
pip install kanren
```

安装成功后，程序开始阶段需导入必要模块并定义辅助函数：

```
from kanren import run, eq, membero, var, conde
from kanren.core import lall
import time
```

2.2 辅助函数设计

在本次实验中，我们需要自定义函数 left、right、next 来表示房子间的相对位置关系。

我们将房子列表转换为元组，使得每个房子都和它旁边的房子对应成元组，然后返回由这些元组组成的列表。此时元组中左边元素即为相邻房子的左侧顺序，使用 kanren 中的 membero，即包含逻辑关系。这样，我们就获得了三个对应的元组，其中包含了这三种简单的逻辑关系。

首先，left() 表示房子 x 在房子 y 的左侧。我们通过将房子列表转化为元组对，然后用 zip(units, units[1:]) 生成相邻房子的元组，使用 membero 来定义这个相邻关系。

接下来，right() 函数通过调换 left() 函数的参数顺序，得到了“右侧”关系。由于 left(x, y, units) 已经表示 x 在 y 的左侧，我们可以直接交换 x 和 y 来表达右侧关系。

最后，next() 函数使用 conde([rules], [rules]))，可以同时判断两种情况：即 x 是否在 y 的左侧，或者 y 是否在 x 的左侧，涵盖了左右相邻的两种可能。通过这种方法，我们同时处理了这两种相邻关系。

```
def left(x, y, units):
    return membero((x, y), zip(units, units[1:]))

def right(x, y, units):
    return left(y, x, units)

def next(x, y, units):
    return conde([left(x, y, units)], [right(x, y, units)])
```

2.3 智能体构建

```
def __init__(self):
    self.units = var()
    self.rules_zebraproblem = None
    self.solutions = None
```

初始化了 units（一个变量），它代表五个房子。
初始化了 rules_zebraproblem，它将存储规则。
初始化了 solutions，它将存储求解结果。

2.4 逻辑规则声明

```
h1, h2, h3, h4, h5 = var(), var(), var(), var(), var()
        units_tuple = (h1, h2, h3, h4, h5)
```

这里，h1, h2, h3, h4, 和 h5 是五个单独的变量，每一个代表一座房子（一座房子包含五个元素：国家、工作、饮料、宠物、颜色）。然后将这五个变量打包成一个元组 units$_tuple$

```
self.rules_zebraproblem = lall(
            # 绑定房子集合
            eq(self.units, units_tuple),

            # 国家与颜色
            membero(('英国人', var(), var(), var(), '红色'), units_tuple),
            # 国家与宠物
            membero(('西班牙人', var(), var(), '狗', var()), units_tuple),
            # 国家与职业
            membero(('日本人', '油漆工', var(), var(), var()), units_tuple),
            # 国家与饮料
            membero(('意大利人', var(), '茶', var(), var()), units_tuple),

            # 第一座房子是挪威人
            eq(h1, ('挪威人', var(), var(), var(), var())),

            # 颜色位置关系：绿色在白色左边（不一定相邻）
            conde([left((var(), var(), var(), var(), '绿色'),
                        (var(), var(), var(), var(), '白色'), units_tuple)],
                  [eq((var(), var(), var(), var(), '绿色'), h3),
                   eq((var(), var(), var(), var(), '白色'), h4)],
                  [eq((var(), var(), var(), var(), '绿色'), h4),
                   eq((var(), var(), var(), var(), '白色'), h5)]),

            # 挪威人旁边是蓝色房子（h1旁边只能是h2）
            eq(h2, (var(), var(), var(), var(), '蓝色')),

            # 职业、宠物、饮料约束
            membero((var(), '摄影师', var(), '蜗牛', var()), units_tuple),
            membero((var(), '外交官', var(), var(), '黄色'), units_tuple),
            membero((var(), '小提琴家', '橘子汁', var(), var()), units_tuple),

            # 中间房子（h3）喝牛奶
            eq(h3, (var(), var(), '牛奶', var(), var())),
            # 绿色房子喝咖啡
            membero((var(), var(), '咖啡', var(), '绿色'), units_tuple),
```

```
            # 存在喝矿泉水的人
            membero((var(), var(), '矿泉水', var(), var()), units_tuple),

            # 存在斑马和马
            membero((var(), var(), var(), '斑马', var()), units_tuple),
            membero((var(), var(), var(), '马', var()), units_tuple),

            # 相邻关系
            next((var(), '医生', var(), var(), var()),
                 (var(), var(), var(), '狐狸', var()), units_tuple),
            next((var(), '外交官', var(), var(), var()),
                 (var(), var(), var(), '马', var()), units_tuple),
        )
```

2.4.1 包含关系 membero

```
membero(('英国人', var(), var(), var(), '红色'), units_tuple),
```

存在一个房子（h1 到 h5 中的某个房子），它的国籍是“英国人”，并且它的颜色是“红色”。其他的属性（如职业、饮料、宠物）暂时不确定，因此用 var() 表示

2.4.2 等价关系 eq

```
eq(h1, ('挪威人', var(), var(), var(), var())),
```

将 h1（第一座房子）绑定为一个包含特定属性的元组，具体来说，它表示 h1 这座房子的国籍是“挪威人”，但其他的属性（职业、饮料、宠物、颜色）尚未确定

2.4.3 和或关系 conde

conde((rules, rules)) 表示逻辑“和”，conde([rules], [rules])) 表示逻辑“或”

2.5 规则求解器

```
def solve(self):
        """
        规则求解器(请勿修改此函数).
        return: 斑马规则求解器给出的答案，共包含五条匹配信息，解唯一.
        """

        self.define_rules()
        self.solutions = run(0, self.units, self.rules_zebraproblem)
        return self.solutions
```

run() 函数通过 self.rules_zebraproblem 中定义的规则进行推理，得到符合条件的所有解。所有解会通过 self.solutions 存储，最终在 solve() 方法中返回。

3 程序代码

```python
from kanren import run, eq, membero, var, conde
# kanren一个描述性Python逻辑编程系统
from kanren.core import lall
# lall包用于定义规则
import time

def left(x, y, units):
    return membero((x, y), zip(units, units[1:]))

def right(x, y, units):
    return left(y, x, units)

def next(x, y, units):
    return conde([left(x, y, units)], [right(x, y, units)])

class Agent:
    """
    推理智能体.
    """
    def __init__(self):
        """
        智能体初始化.
        """
        self.units = var()
        self.rules_zebraproblem = None
        self.solutions = None

    def define_rules(self):
        """
        定义逻辑规则.
        """
        h1, h2, h3, h4, h5 = var(), var(), var(), var(), var()
        units_tuple = (h1, h2, h3, h4, h5)
        # self.units共包含五个unit成员
        # 即每一个unit对应的var都指代一座房子(国家, 工作, 饮料, 宠物, 颜色)
        # 各个unit房子又包含五个成员属性: (国家, 工作, 饮料, 宠物, 颜色)

        self.rules_zebraproblem = lall(
            # 绑定房子集合
            eq(self.units, units_tuple),

            # 国家与颜色
            membero(('英国人', var(), var(), var(), '红色'), units_tuple),
            # 国家与宠物
            membero(('西班牙人', var(), var(), '狗', var()), units_tuple),
            # 国家与职业
```

```
            membero(('日本人', '油漆工', var(), var(), var()), units_tuple),
            # 国家与饮料
            membero(('意大利人', var(), '茶', var(), var()), units_tuple),

            # 第一座房子是挪威人
            eq(h1, ('挪威人', var(), var(), var(), var())),

            # 颜色位置关系：绿色在白色左边（不一定相邻）
            conde([left((var(), var(), var(), var(), '绿色'),
                        (var(), var(), var(), var(), '白色'), units_tuple)],
                  [eq((var(), var(), var(), var(), '绿色'), h3),
                   eq((var(), var(), var(), var(), '白色'), h4)],
                  [eq((var(), var(), var(), var(), '绿色'), h4),
                   eq((var(), var(), var(), var(), '白色'), h5)]),

            # 挪威人旁边是蓝色房子（h1旁边只能是h2）
            eq(h2, (var(), var(), var(), var(), '蓝色')),

            # 职业、宠物、饮料约束
            membero((var(), '摄影师', var(), '蜗牛', var()), units_tuple),
            membero((var(), '外交官', var(), var(), '黄色'), units_tuple),
            membero((var(), '小提琴家', '橘子汁', var(), var()), units_tuple),

            # 中间房子（h3）喝牛奶
            eq(h3, (var(), var(), '牛奶', var(), var())),
            # 绿色房子喝咖啡
            membero((var(), var(), '咖啡', var(), '绿色'), units_tuple),
            # 存在喝矿泉水的人
            membero((var(), var(), '矿泉水', var(), var()), units_tuple),

            # 存在斑马和马
            membero((var(), var(), var(), '斑马', var()), units_tuple),
            membero((var(), var(), var(), '马', var()), units_tuple),

            # 相邻关系
            next((var(), '医生', var(), var(), var()),
                 (var(), var(), var(), '狐狸', var()), units_tuple),
            next((var(), '外交官', var(), var(), var()),
                 (var(), var(), var(), '马', var()), units_tuple),
        )

    def solve(self):
        """
        规则求解器(请勿修改此函数).
        return: 斑马规则求解器给出的答案, 共包含五条匹配信息, 解唯一.
        """

        self.define_rules()
        self.solutions = run(0, self.units, self.rules_zebraproblem)
```

```
        return self.solutions

agent = Agent()
solutions = agent.solve()

# 提取解释器的输出
output = [house for house in solutions[0] if '斑马' in house][0][4]
print ('\n{}房子里的人养斑马'.format(output))
output = [house for house in solutions[0] if '矿泉水' in house][0][4]
print ('{}房子里的人喜欢喝矿泉水'.format(output))

# 解释器的输出结果展示
for i in solutions[0]:
    print(i)
```

4 测试结果

在线测试 & 平台测试

```
绿色房子里的人养斑马
黄色房子里的人喜欢喝矿泉水
('挪威人', '外交官', '矿泉水', '狐狸', '黄色')
('意大利人', '医生', '茶', '马', '蓝色')
('英国人', '摄影师', '牛奶', '蜗牛', '红色')
('日本人', '油漆工', '咖啡', '斑马', '绿色')
('西班牙人', '小提琴家', '橘子汁', '狗', '白色')
```



5 总结

在本次实验中，我初次使用了 kanren 包进行逻辑推理，学习和应用 membero，eq，conde 等逻辑关系，并借助其得到自定义函数 left、right、next 来表示房子之间的相对位置关系从而实现规则的表述，最终通过推理得到结果。